

## Agenda

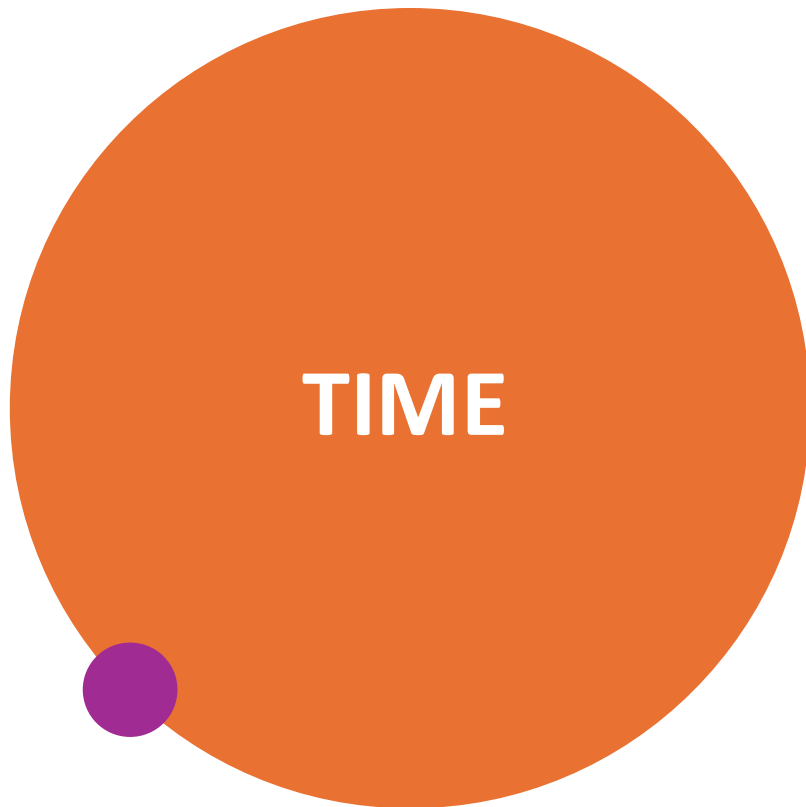
- This Week
  - Class 11 – October 14<sup>th</sup> (2:00 – 3:00, 3:30 – 4:30)
  - Class 12 – October 15<sup>th</sup> (10:30 – 12:30)
  - Class 13 – October 16<sup>th</sup> (1:00 – 3:00)
- Starter Quiz
- Data Architecture
- Thought Paper Presentation – AI in Financial Fraud Prevention and AML (Team 12) - 30 mins
- Data Architecture (Cont'd)

### Note:

- Mobile Phones – Do not bring to class or Completely Switch off
- If students have any specific topic they want to discuss, share the topic and we can try to schedule
- Please be seated by your teams

## Starter Quiz

- What is the resource we all have **that is finite and is always in depleting quantities**



- The currency of life is time
- You can invest it, spend it, or waste it.
- Go splurge and waste your money if you want, but wasting time is something U would regret later

## **Starter Quiz**

- **What would be a Use Case When an Account moves from Liability to an Asset or Vice Versa**



# Overdraft

- Current / Checking Account with Positive Balance is an Liability (i.e. The Bank Owes Money to Customer)
- If this account has an Overdraft Facility and that gets used (when the account does not have enough balance to cover a Debit Payment) then the Account becomes an Asset as the Customer Owes Money to the Bank)

# Core Banking Platform Engineering – Data Architecture

# Data Architecture – Topics Covered

- **Data Model**
    - Logical Model vs Physical Model
    - FSDM – Industry Reference Model
  - **Databases needed for Core Banking Platform Engineering**
    - Transactional Database
    - Reporting and Analytical Database
  - **Key Data Design Principles and Considerations**
    - SQL Standards
    - Designing for SBCS and DBCS
    - Designing for Flexible Amount Formats
    - Designing for Multiple External ID's
    - Data Change History and Audit
    - Data Lineage
    - Data Archiving
    - Data Privacy
- Note: Data Security will be handled in the Security Architecture Slides

# Data Model – Logical Model Vs Physical Model



# Overview of Data Models

## Role of Data Models

Data models organize and structure data for efficient storage, retrieval, and manipulation in databases.

## Logical Data Model

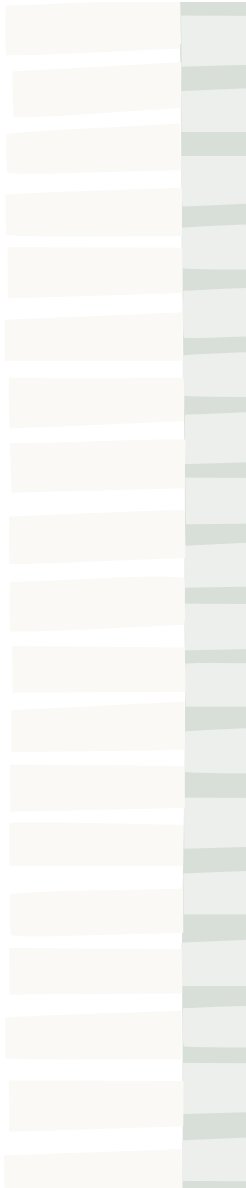
Represents abstract data structure focusing on entities, attributes, and relationships without technical details.

## Physical Data Model

Details database schema including tables, columns, data types, indexes, and storage mechanisms.

## Importance of Understanding Models

Knowing differences between models helps design scalable, efficient, and maintainable databases.



# Purpose and Focus

## Logical Data Model Purpose

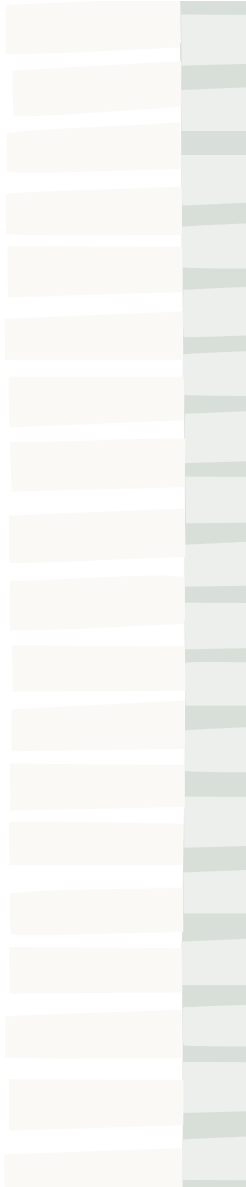
Represents business requirements and data relationships conceptually, independent of database technology.

## Physical Data Model Focus

Details tables, columns, data types, indexes, and storage formats for database implementation.

## Blueprint to Implementation

Logical model serves as blueprint, physical model translates it into deployable database schema.



# Audience and Platform Dependency

## Logical Data Models

Logical models focus on business requirements and are platform-independent, allowing flexible database choices during implementation.

## Physical Data Models

Physical models are platform-specific, optimized for chosen database systems like Oracle or DB2 or Postgres or SQL Server, ensuring operational efficiency.

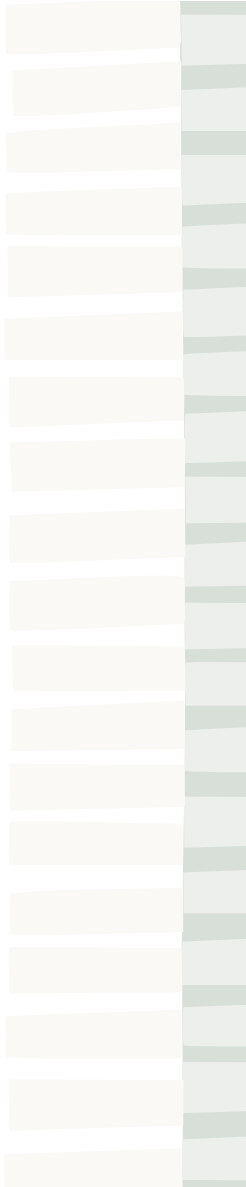
# Validation and Examples

## Logical Data Model Validation

Logical models ensure business logic is captured and data relationships align with requirements.

## Physical Data Model Validation

Physical models ensure database performance, scalability, and data integrity through proper structure.



## Included Elements and Use Cases

### **Logical Data Model Elements**

Logical models include entities, attributes, relationships, and business rules defining data structure and interactions.

### **Physical Data Model Elements**

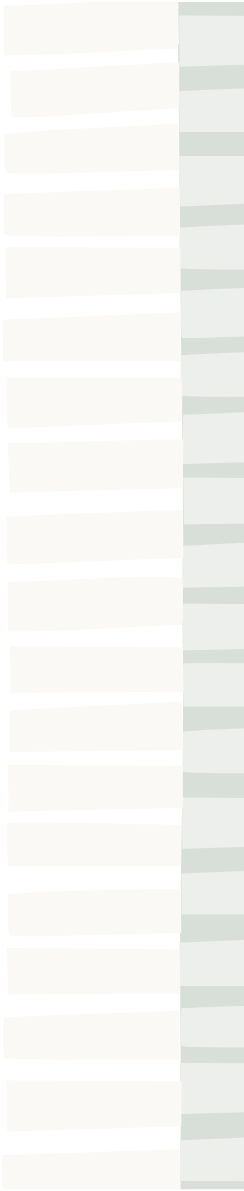
Physical models include tables, columns, data types, indexes, foreign keys, and storage formats for implementation.

### **Use Cases of Logical Models**

Logical models are used in early design stages for planning and validating business requirements accurately.

### **Use Cases of Physical Models**

Physical models guide database deployment, optimization, and maintenance for performance and scalability.



# ER Diagrams

## Modelling Database Structure

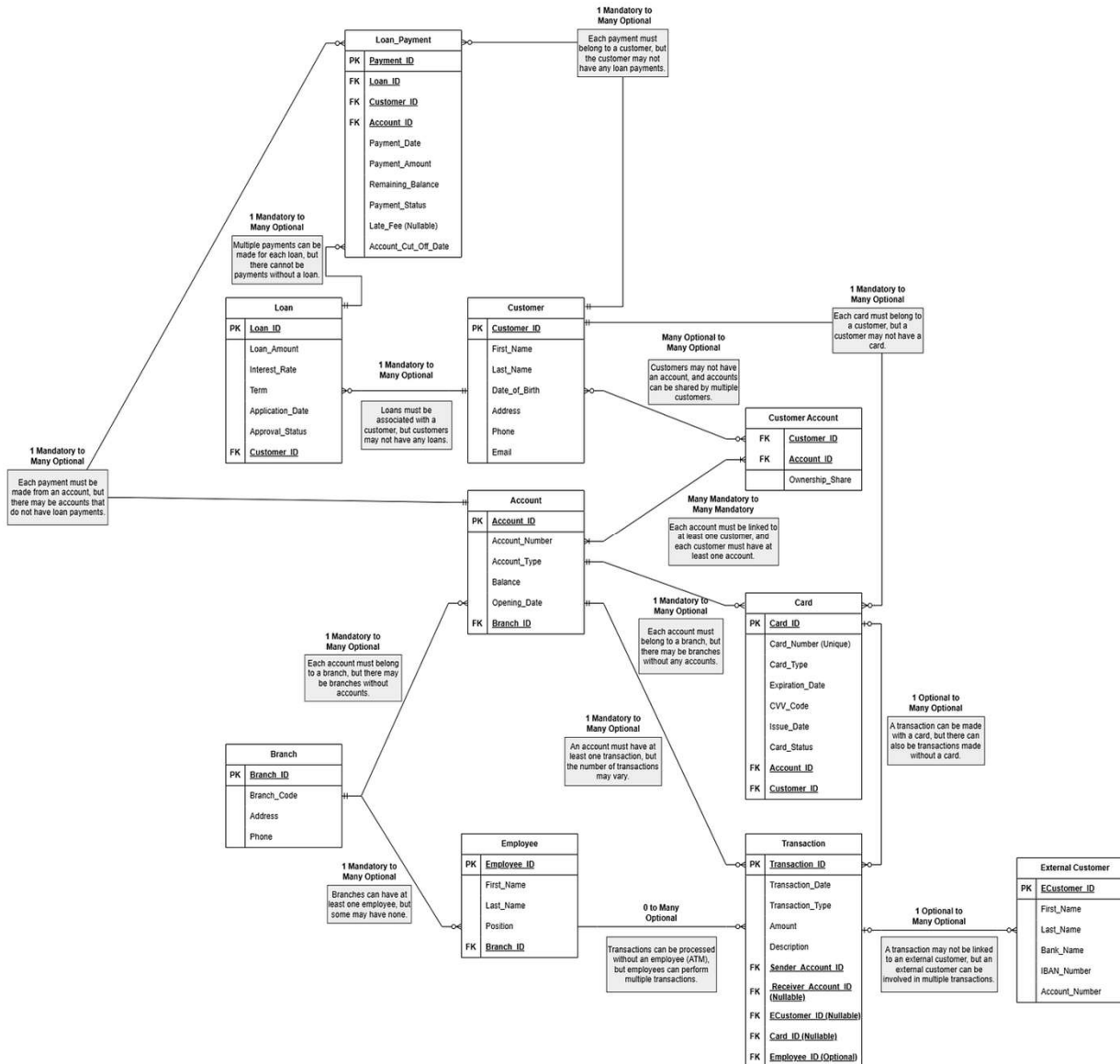
ER Diagrams visually represent how data is structured and related within an system.

## Entities and Attributes

Entities like Customer each contain specific attributes such as Name, ID, Title, Gender.

## Relationships Between Entities

Relationships such as Account to Customer Relationships link Account to Customers showing the interaction between different entities



# Popular Data Modelling Tools

## ER/Studio for Enterprises

ER/Studio is designed for complex enterprise environments, enabling detailed visualization and management of large data models.

## IBM Infosphere

IBM Infosphere Data Architect facilitates collaborative data modelling, supporting teams working together on structural data designs.

## Microsoft Visio for Simple Diagrams

Microsoft Visio is a popular choice for diagramming simple data models, offering an accessible interface for creating clear visual representations.



# IBM FSDM Logical Data Model Overview

## Industry-Specific Data Framework

FSDM offers a tailored framework for financial institutions, aligning data structures with industry requirements and best practices.

## Robust Data Framework

FSDM provides a structured framework for enterprise data management in financial services, enhancing data governance.

## Standardization and Compliance

The model standardizes data definitions, supporting regulatory compliance while boosting the consistency and quality of organizational data.

## Physical Implementation Independence

Designed to be independent of physical platforms, enabling flexible deployment across technologies.



# Transactional Databases





# Transactional Databases

## **Efficient Transaction Processing**

Transactional databases are optimized for managing large volumes of real-time transactions with speed and reliability.

## **ACID Properties**

ACID properties ensure every transaction is processed with atomicity, consistency, isolation, and durability for data reliability.

## **Industry Applications**

Transactional databases are vital for banking, e-commerce, and business applications, supporting rapid data changes with integrity.

# Transactional Databases – RDBMS

## **Structured Data Management**

RDBMS platforms are designed to efficiently store and organize structured data, supporting various business applications.

## **SQL Query Support**

These systems support SQL, enabling users to query and manipulate data quickly and accurately.

## **Data Integrity and Scalability**

RDBMS platforms offer robust transaction management, ensuring data integrity and the ability to scale for growing business needs.



# Popular RDBMS Examples

## **Postgres**

Postgres is an open-source RDBMS widely used in due to its low cost, reliability and flexibility.

## **SQL Server**

SQL Server from Microsoft is well-known for its smooth integration with Microsoft products and tools, making it popular in business environments.

## **Oracle**

Oracle Database offers strong scalability and robust features, making it a top choice for large enterprise environments that run on Open Systems (e.g.. Unix, Linux).

## **DB2**

Database from IBM widely used in Mainframe based applications



# Reporting and Analytics Databases

# PURPOSE & DESIGN OF ANALYTICAL DATABASES

## **Optimized for Complex Queries**

Analytical databases prioritize fast querying and data analysis over transactional operations like inserts and updates.

## **Columnar Storage Format**

They use columnar storage to enable efficient data compression and quick retrieval for read-heavy workloads.

## **Distributed Architecture**

Built on distributed systems, these databases scale horizontally managing vast data across multiple nodes.

## **Support for Real-Time Analytics**

Designed to generate Reports, real-time analytics, helping organizations make quick, data-driven decisions.

# KEY PLATFORMS

## **ClickHouse**

ClickHouse is an open-source columnar database optimized for real-time analytics and fast query performance.

## **Hadoop and Apache Kylin**

Hadoop provides distributed storage and processing, with Apache Kylin enabling OLAP and big data multidimensional analysis.

## **DataBricks**

DataBricks integrates Apache Spark for a unified analytics platform, supporting scalable data engineering and machine learning workloads.

## **Snowflake**

Snowflake is a cloud-native data warehouse known for performance, scalability, and ease of use in cloud environments.

# KEY CONSIDERATIONS

## **Scalability and Performance**

Systems must handle extremely high transaction volumes with efficient scalability and optimized performance.

## **Security and Compliance**

Embedded encryption and secure deployment protocols are critical to protect sensitive data and ensure compliance.

## **Observability and Monitoring**

Real-time monitoring and anomaly detection tools maintain system health and improve resilience.

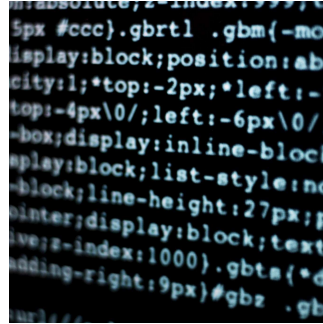
## **AI Integration**

Integrating large language models and automation tools enhances data interaction and drives continuous innovation.

# Key Data Design Principles and Considerations for Engineering Global Core Banking Platforms



# SQL Standards



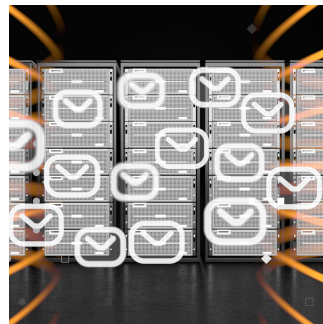
## Unified Query Language

SQL standards provide a common language for querying and managing data across different relational databases, simplifying development and maintenance.



## Cross-Database Compatibility

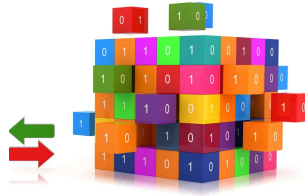
Following ANSI SQL standards enables seamless migration and integration between databases, reducing vendor lock-in and improving flexibility.



## Core Compliance Importance

Despite vendor-specific SQL extensions, maintaining compliance with core standards is essential for supporting multiple platforms.

# Designing for SBCS & DBCS



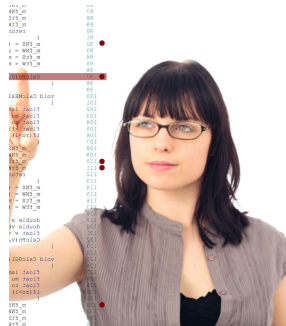
## Character Set Compatibility

Supporting both SBCS and DBCS is crucial for databases to handle international languages and characters effectively.



## Proper Encoding Selection

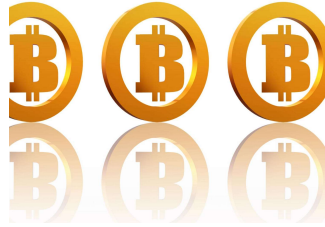
Choosing appropriate encodings like UTF-8 or UTF-16 ensures accurate and efficient data storage for multilingual databases.



## Accommodating Variable-Width Characters

Database field lengths and indexing strategies must be designed for variable-width characters to avoid data truncation.

# Designing for Flexible Amount Formats



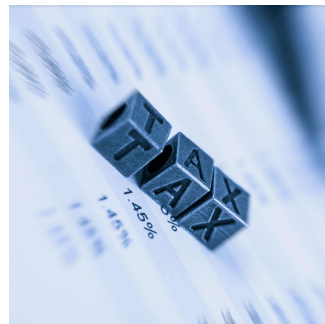
## Versatile Currency Support

Financial systems must handle both national currencies and stablecoins, requiring adaptable data formats for each type.



## Accommodating Decimal Variations

Different currencies use different decimal places, so systems should accurately represent values without loss or error.



## Clear Asset Identification

It's vital to clearly distinguish between fiat and digital asset codes for seamless processing and regulatory compliance.



# Currency Amount and Rate Formats

---

## Decimal Precision

National currencies can be of Zero Decimal Precision, Two Decimal Precision or Three Decimal Precisions.

Digital currencies like Major Stablecoins (e.g.. USDC, USDT) have 6 Decimal Precision, though Cryptocurrencies can have a can be more (up to 18 decimals).

## Decimal Formatting

Processing: Accurate Decimal formatting and Precision handling is needed to ensure transaction consistency and prevents rounding errors across digital platforms.

## Database Design:

**Amount Fields:** Have the Precesion to support all National Currencies and Digital Currencies

**Exchange Rates:** Most FX Markets typically deal with Exchange Rate having 4 Decimal Precision, but there are cases when more are needed and generally most platforms design with 10 Decimal Precision

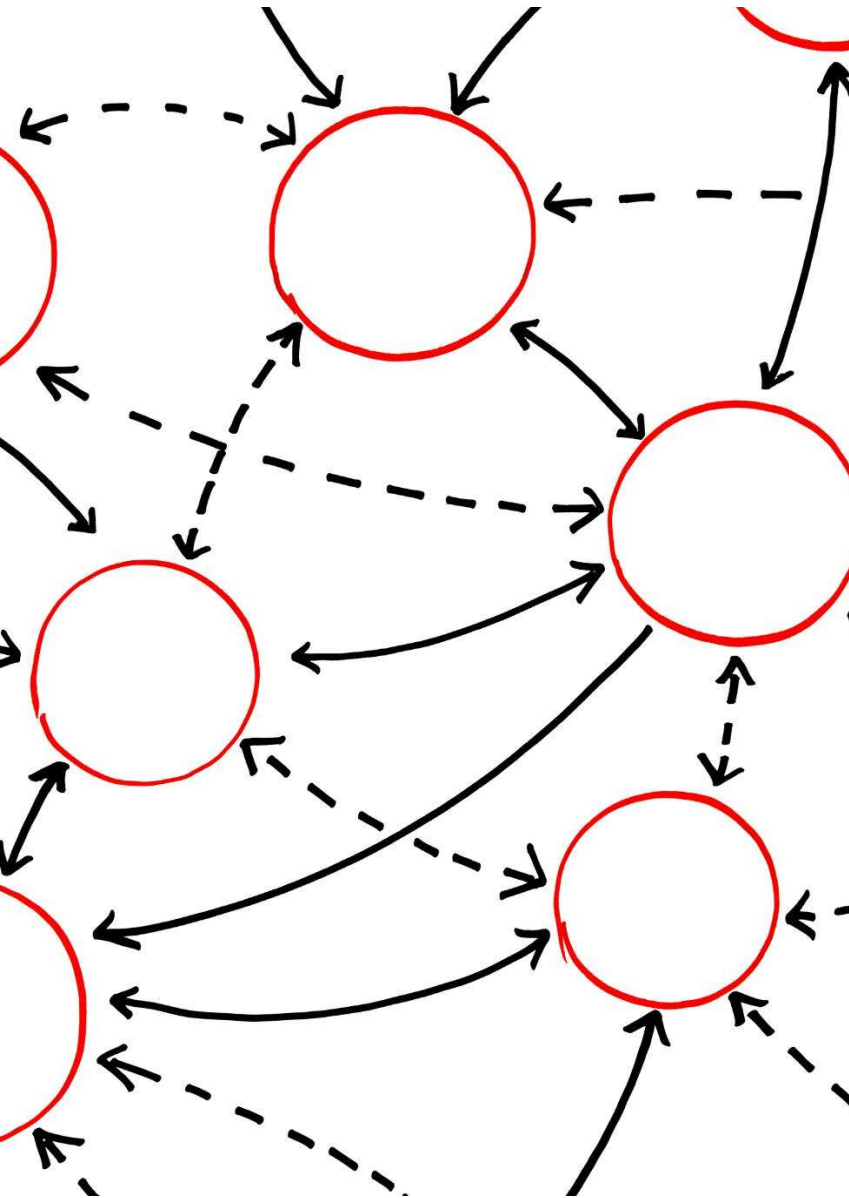
# Designing for Multiple External Account ID's

## Foundational ID , Functional ID Concept

- Relate Multiple External Account ID's also known as Functional ID's (e.g.. Account Number, IBAN Number, CLABE in Mexico..) to a **Unique and Non-Changeable “Foundational ID”** (could be Application Internally Generated ID for an Account Number if this is used for other purposes such as Database Partitioning) . Example of such an Entity:
  - Internal ID (e.g. Generated by Application)
  - Ext Account ID Type (e.g.: Account Number, IBAN, CLABE)
  - Ext Account ID Value

*Note: Reference the example of India's approach using similar concept with Aadhar as Foundational ID and Multiple Functional ID's like PAN, DL, Passport, Voter Idetc. related to it.*

- All Published Interfaces (Incoming or Outgoing) will have External ID Type / Value fields only, and the Application will translate that to Internal ID as a first step and then use the Internal ID in the rest of the processing
- All other Entities (i.e. Database Tables) within the Application will use Internal ID only as the Key / Attribute.



# Data Lineage

## Tracking Data Flow

Data lineage tracks how data moves through systems, revealing its origins, transformations, and ultimate usage for better understanding.

## Ensuring Transparency and Quality

By visualizing data lineage, organizations can trace errors and maintain high data quality

## Supporting Compliance and Governance

Understanding data lineage is essential for compliance, enabling efficient auditing and governance of crucial information assets.

## Facilitating Impact Analysis

Understanding data lineage helps organizations assess the impact of changes, leading to informed decision-making and enhanced data quality management.



# Capturing Data Lineage

## Documenting Data Sources

Start by recording all data sources and tracking every stage of data transformation within workflows for clarity.

## Tools

Employ metadata platforms to automatically log and visually represent how data moves through systems.

## Maintenance

Keep data lineage continuously updated to support transparency, regulatory compliance, and easier troubleshooting in data processes.



# Data Archiving



## Long-Term Information Storage

Data archiving securely preserves inactive information for retention and compliance over extended periods.



## Reducing Storage Costs

Archiving rarely accessed data moves it to cost-effective storage, helping organizations save resources and manage data efficiently.



## Quick and Safe Retrieval

Effective data archiving provides rapid access to stored information and protects it from loss or deletion.



# Key Data Archiving Principles

## **Clear Retention Policies**

Data archiving requires well-defined retention policies to determine what information is kept and for how long.

## **Classification and Indexing**

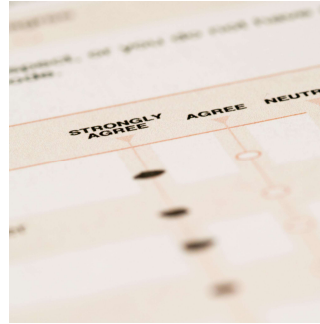
Accurate classification and indexing of archived data makes retrieval quick and efficient whenever the information is needed.

## **Security and Compliance**

Robust access controls and compliance with legal regulations are essential for secure and lawful data storage and privacy.



# Data Changes History and Audit



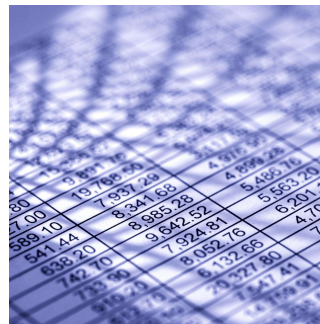
## Processing Necessity in Core Banking

Recording data modification in banking systems is a must for processing needs (e.g.. Processing Backdated Transactions).



## Regulatory Compliance Support

Maintaining data change logs helps satisfy legal and regulatory obligations, making it easier to demonstrate compliance.



## Efficient Troubleshooting and Investigation

Audit trails enable quick detection of unauthorized changes and simplify troubleshooting during audits or system investigations.

# Data Changes History and Audit – Design Patterns

## Insert Only Design Pattern

### **Immutable Data Storage**

Records are never altered or deleted, guaranteeing that original data remains unchanged throughout its lifecycle.

### **Enhanced Audit and Analysis**

Maintaining all historical records allows for robust audit trails and comprehensive historical data analysis.

### **Scalability and Efficiency**

Append-only databases enable efficient partitioning and replication, improving scalability for large systems.



# Data Changes History and Audit – Design Patterns

## Insert Only Design Pattern

- In Insert Only Design Pattern all Changes are handled as Inserts in Tables through a CRUD Column and including Effective Date and Timestamp as a part of the Physical Unique e.g.
  - Data Create: Inserted with CRUD is “C”
  - Data Update: Inserted with CRUD as “U”
  - Data Delete: Inserted with CRUD as “D”
- Data is Queried using Effective Date and Timestamp
- Table Indexes include Effective Date and Timestamp
- Audit Attributes to be captured:
  - Who - User ID
  - Where - Channel ID, Device ID, Process ID
  - When - Initiated TS, Accepted TS, Processed TS



# Data Privacy

## **Purpose of Data Privacy Laws**

Data privacy regulations protect personal information by setting standards for organizations on how data is handled and secured.

## **Organizational Responsibilities**

Businesses must adopt strict measures for collecting, storing, and processing personal data to remain compliant with regulations.

## **Consequences of Non-Compliance**

Failure to comply can lead to heavy fines, loss of reputation, and diminished consumer trust, impacting business success.





# Global Data Privacy Laws

## **DPDPA in India**

India's data privacy law is the DIGITAL PERSONAL DATA PROTECTION ACT, a comprehensive framework enacted in August 2023 to protect citizens' digital personal data and establish rules for its processing by businesses and organizations.

## **GDPR in Europe**

The GDPR sets strict rules for personal data handling, requiring organizations to protect individual privacy and offer transparency.

## **CCPA in California**

The CCPA gives California consumers more control over their personal data and enhances transparency in data practices.

